

PatentPincer

An autonomous patentability analyst over SerpApi prior art

Dipankar Sarkar

github.com/doom2quake

Abstract

A prior-art search is the first thing a founder needs and the last thing they can afford: a patent attorney charges four figures and takes a week to say whether an idea is already patented. PatentPincer collapses that first search into a minute. Describe an invention in one sentence and it searches the prior art, fetches the numbered claims of the closest references, compares the invention element by element, and returns a decision-ready verdict (CLEAR, NEEDS_REVIEW, HIGH_OVERLAP, or SEARCH_UNAVAILABLE) with the closest reference and a next step. SerpApi's Google Patents, Patents Details and Scholar engines are the load-bearing data backbone; the analyst is a skill graph on the agent-core, with API spend held behind a guardrail and the verdict routed to the right destination. The design commitment is that the system refuses rather than guesses: a search that did not succeed, a reference set that came back empty, or claim text that was never retrieved all fail closed, and never to a clearance.

1 Problem

Most founders skip the prior-art search and find out the expensive way, after they have built. The information exists (patents are public, papers are indexed) but assembling it, reading it against a specific claim, and turning it into a file-or-narrow-or-walk decision is exactly the slow, costly step. The evidence is also split: a claim can be anticipated by a granted patent or by non-patent literature, and a search that reads only one of the two misses half the art. We want that first pass to be fast, cited, and honest about how close the nearest reference sits.

2 Design

PatentPincer runs the workflow the way an attorney actually works, as named skills assembled into an ADK supervisor that delegates in order:

```
1 search-prior-art (Patents + Scholar)
2 fetch claim text (Patents Details)
3 assess-novelty (element-by-element)
4 render-verdict (CLEAR|NEEDS_REVIEW|
                  HIGH_OVERLAP|
                  SEARCH_UNAVAILABLE)
```

The searcher varies its phrasing across a few Google Patents queries to widen recall and adds one Scholar query for non-patent literature. It then calls the Google Patents Details engine for the shortlist, because a search snippet is not a claim and an element-by-element comparison against snippets is guessing. The analyst decomposes the invention into claim elements and scores each element against every numbered claim of every reference; a cell records the reference, the claim number, the terms that matched and the excerpt they matched in, so every judgement is checkable and overrutable. A single reference disclosing at least 80% of the elements anticipates (HIGH_OVERLAP); at least 40% disclosed jointly is a combination risk (NEEDS_REVIEW). The brief begins with a machine-readable VERDICT: line so the outcome can be routed.

3 Refusing rather than guessing

The failure that matters in this product is a confident clearance built on nothing, so four rules are enforced in the pipeline and pinned by tests. If no Google Patents search succeeded (guardrail suppression, timeout, bad key, quota) the verdict is SEARCH_UNAVAILABLE and an alert is raised. If searches succeeded but returned zero references, the verdict is at best NEEDS_REVIEW: an empty reference set scores zero of N elements disclosed, which would otherwise read as the strongest possible CLEAR off the weakest possible evidence. If references were found but no claim text came back, there is nothing to compare and the run refuses. And if the references came from the offline demo corpus rather than a live SerpApi response, a computed CLEAR is downgraded to NEEDS_REVIEW, because fixture data can warn you off an idea but cannot clear one. Provenance is carried on every record and every call (ok, fixture, fixture_unavailable, suppressed, error); only ok is evidence.

4 The hero: SerpApi as the load-bearing backbone

The product is only as good as its prior-art coverage, and that coverage is SerpApi. Google Patents returns structured, current patent records (assignee, priority date, links) that are impractical to scrape reliably; the Patents Details engine returns the numbered claims the comparison actually reads; and Scholar catches the literature that anticipates just as many claims. Remove SerpApi and there is no evidence left to reason over, which is precisely why the pipeline refuses to emit a verdict when a SerpApi call did not succeed. Spend is bounded rather than trusted: every call passes an ActionLimiter check before it leaves the process, and the per-cycle cap is sized to exactly one assessment (query variants, one Scholar query, and the claim fetches), so a runaway loop cannot burn credits. A budget that starves the claim fetch would silently degrade the analysis, so the optional LLM narrative is given its own spend cycle and cannot take the analyst's. The three tools are also exposed over MCP with an explicit `run_id` argument, since a separate process cannot see the caller's run context. The verdict is then routed on a strict parse of the first line: `HIGH_OVERLAP` and `SEARCH_UNAVAILABLE` raise an alert and open a ticket; `NEEDS_REVIEW` and `CLEAR` open a ticket; an unparseable brief goes to a human rather than being guessed at.

5 Results

The test suite (pytest, 38 tests) runs with no network and no API key and covers each rule the product depends on. Provenance: offline records are labelled fixture and never ok, queries outside the demo corpus are refused instead of answered with someone else's patents, the before and assignee filters really apply, an unparseable cutoff is an error, and a live response carrying its own status field cannot relabel a call. Claim text: the Details engine yields numbered claims and matrix cells quote them verbatim. Fail-closed: dry-run suppression, timeout, an API-level error body, zero results, and references with no claim text each refuse rather than clear, and a fixture `CLEAR` is downgraded while a live `CLEAR` is not. Router: a `CLEAR` brief that contains the phrase "high overlap" still routes as a clearance, and a malformed verdict line routes to a human. Spend and context: concurrent assessments keep separate ledgers and budgets, and an explicit `run_id` beats the ambient context. The live SerpApi branch is exercised through a faked `urlopen` covering success, error bodies, timeouts, empty results and missing claims.

6 Related work and limitations

Automated prior-art retrieval and patentability scoring are established; the contribution here is the end-to-end analyst path (structured SerpApi retrieval, claim-text retrieval, element-by-element comparison, a routed verdict) packaged so it runs in a minute, with explicit provenance and refusal semantics. The comparison itself is deliberately plain: light suffix stemming, stopword removal and term containment of an element in a claim, a first-pass screen rather than a trained model or a legal opinion. Its value is that it shows its work, so a reviewer can see which words carried a match and overrule the cell. We report no live-key measurements: this build had no funded SerpApi key and no GCP project, so no recorded run carries LIVE provenance and no LLM narrative was generated; the live path is covered by fakes, which validates request construction, parsing and labelling but not real retrieval quality. Coverage is in any case bounded by what Patents and Scholar index, and PatentPincer informs a filing decision rather than replacing counsel.

7 Conclusion

The first prior-art pass does not have to cost a week and four figures. PatentPincer turns a one-sentence invention into a cited, decision-ready verdict by treating SerpApi as the evidence and the agent as the analyst that reasons over it.